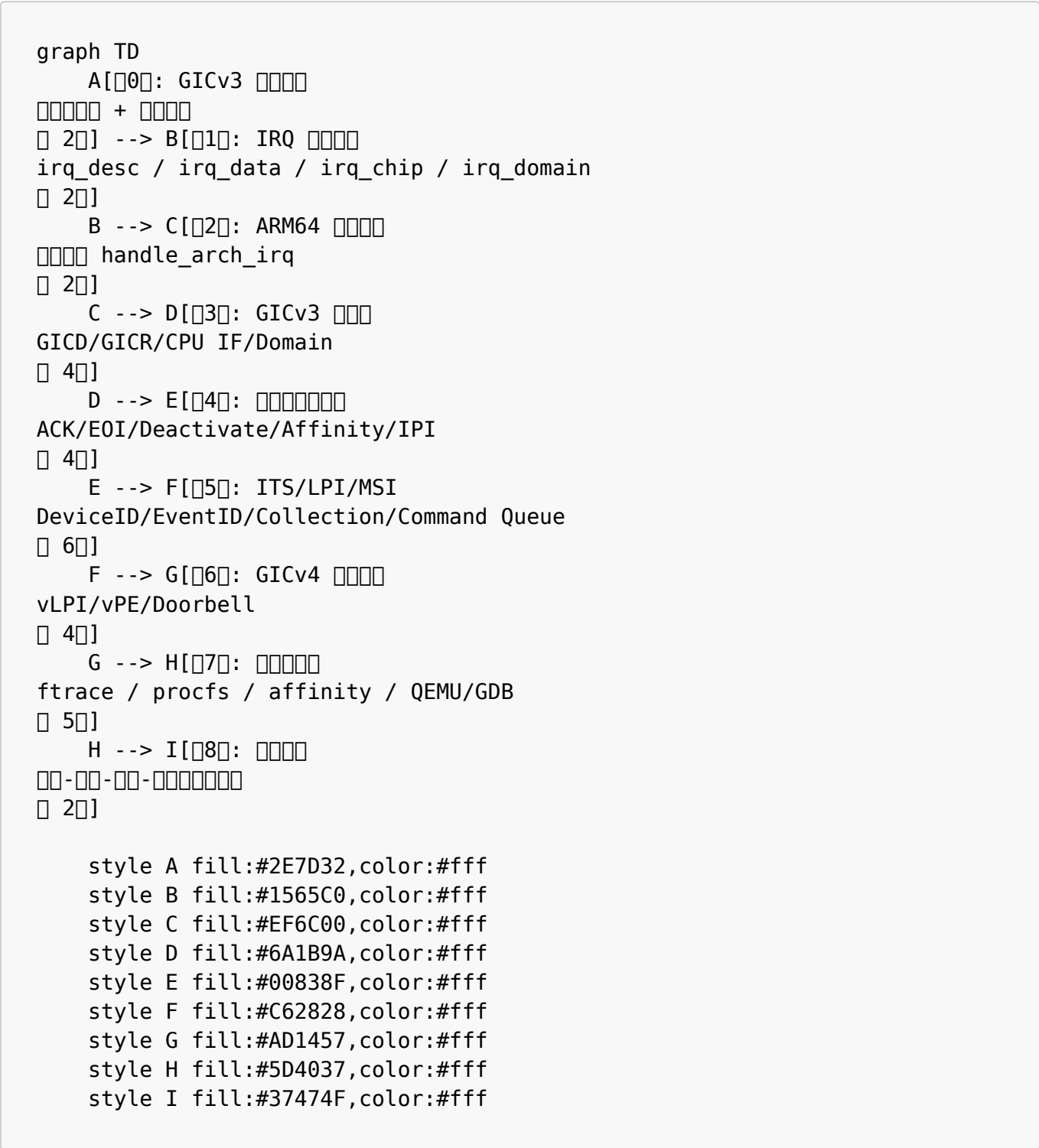


Linux ARM64 GICv3

Linux 6.18.1 / ARM64 / ARMv8 / GICv3 / ITS / GICv4



= =Linux = = = =ITS/MSI = =

00000000 GICv3 00000000

0000000000000000 GICv300000000“00000000”0000000000

1. **ARM GIC** 0000Distributor0Redistributor0CPU Interface0ITS0vPE/vLPI 000000000000
2. **Linux** 00 **IRQ** 0000irq_desc0irq_data0irq_chip0irq_domain 000“000000”0“Linux IRQ 0 0”000
3. 0000000000drivers/irqchip/irq-gic-v3.c0irq-gic-v3-its.c0irq-gic-v4.c 0000000000 Linux IRQ core 0000000000

00000000000000

1. 00000000 **ISR** 00000000
2. 0000000000 **GICD/GICR/ICC sysreg** 0000
3. **irq_domain** 000 **hwirq** 000 **virq**
4. **ITS** 000 **DeviceID/EventID** 000 **LPI**00000000000000
5. **GICv4** 000000 **LPI** 000 **guest** 000 **vLPI/vPE**

00000000000000 ARM64 GICv3 0000000000“00”0000“0000000000000000000000000000000000”0

0000GICv3 0000

0.1 000000

```

00/00 -> SPI/ESPI -----
                                |
CPU000 -> SGI/PPI/EPPI -> |
                                |
PCIe/MSI -> ITS -> LPI -----> GIC 00 -> CPU Interface(ICC_*) -> ARM64
IRQ 0000

```

000

- GICD 00000000SPI / ESPI 000000
- GICR 0000 CPU 000000SGI / PPI / EPPI / LPI 0000
- ICC_* system registers 0 CPU 000000
- ITS 00 MSI/LPI 00
- GICv4 00000000 vLPI / vPE

0.2 000000000000

00000000drivers/irqchip/irq-gic-v3.c 00 __get_intid_range() 00000000

1. **SGI_RANGE**00-1500000000
2. **PPI_RANGE**0016-3100 CPU 00000000
3. **SPI_RANGE**0032-10190000000000
4. **EPPI_RANGE**00 **EPPI_BASE_INTID=1056** 000000 PPI
5. **ESPI_RANGE**00 **ESPI_BASE_INTID=4096** 000000 SPI
6. **LPI_RANGE**008192 0000ITS 0000 Locality-specific Peripheral Interrupt

0.2 GICv3 0.2.1 GICv3 0.2.1

1. GICD GICR
2. affinity
3. flow handler
4. ITS/LPI

0.3 GICv3

arm64-reference/gic/IHI0069H_b_gic_architecture_specification.pdf Linux

1. Inactive
2. Pending
3. Active
4. Active+Pending

Linux

1. ICC_IAR1_EL1 ACK INTID
2. ICC_EOIR1_EL1 drop
3. ICC_DIR_EL1 GICD/GICR ICACTIVER deactivate

EOI mode 1 priority drop deactivate supports_deactivate_key gic_eoimodel_chip gic_eoimodel_eoi_irq()

0.4

arm64-reference/gic PDF

1. IHI0069H_b_gic_architecture_specification.pdf Distributor Redistributor Interrupt state System register interface ITS GICv4
2. GICv3_Software_Overview_Official_Release_B.pdf
3. GICv3_v4_overview.pdf GICv3 GICv4
4. learn_the_architecture_-_generic_interrupt_controller_v3_and_v4__overview_198123_0302_03_en.pdf

1. IHI0069
2. Linux
3. Linux

1 Linux IRQ

IRQ core GIC

1.1 irq_desc

```

#include/linux/irqdesc.h

```

Linux IRQ

1. `irq_data`
2. `handle_irq` flow handler
3. `action` ISR
4. `lock`
5. `percpu_enabled / affinity_hint` per-cpu

`irq_desc` Linux

1.2 `irq_data`

```

#include/linux/irq.h

```

1. `irq` Linux IRQ
2. `hwirq`
3. `chip`
4. `domain` `hwirq` `virq`
5. `chip_data`

`irq_data` Linux IRQ

1.3 `irq_chip`

1. `irq_mask`
2. `irq_unmask`
3. `irq_eoi`
4. `irq_set_type`
5. `irq_set_affinity`
6. `irq_set_vcpu_affinity`
7. `ipi_send_mask`

GICv3

1. `gic_chip`
2. `gic_eoimodel_chip`
3. `its_irq_chip`
4. `mbi_irq_chip`

`irq_chip`

1.4 `irq_domain`

```

#include/linux/irqdomain.h

```

/ACPI Linux `virq`

操作

- 1. `translate`
- 2. `alloc`
- 3. `free`
- 4. `select`
- 5. `map`

GICv3 操作 `gic_irq_domain_ops` ITS 操作 GIC domain 操作 MSI domain

操作 `irq_domain` Linux “操作”操作

2 ARM64 操作 GIC

2.1 操作

ARM64 操作 `arch/arm64/kernel/entry.S` “IRQ 操作”操作

操作 `vectors`

```
SYM_CODE_START(vectors)
    EL1t: sync / irq / fiq / error
    EL1h: sync / irq / fiq / error
    EL0 64-bit: sync / irq / fiq / error
    EL0 32-bit: sync / irq / fiq / error
SYM_CODE_END(vectors)
```

操作 `VBAR_EL1` 操作 **16** 操作 4 操作

操作	sync	irq	fiq	error
EL1t	<code>kernel_ventry 1, t, 64, sync</code>	<code>kernel_ventry 1, t, 64, irq</code>	<code>kernel_ventry 1, t, 64, fiq</code>	<code>kernel_ventry 1, t, 64, error</code>
EL1h	<code>kernel_ventry 1, h, 64, sync</code>	<code>kernel_ventry 1, h, 64, irq</code>	<code>kernel_ventry 1, h, 64, fiq</code>	<code>kernel_ventry 1, h, 64, error</code>
EL0 64-bit	<code>kernel_ventry 0, t, 64, sync</code>	<code>kernel_ventry 0, t, 64, irq</code>	<code>kernel_ventry 0, t, 64, fiq</code>	<code>kernel_ventry 0, t, 64, error</code>
EL0 32-bit	<code>kernel_ventry 0, t, 32, sync</code>	<code>kernel_ventry 0, t, 32, irq</code>	<code>kernel_ventry 0, t, 32, fiq</code>	<code>kernel_ventry 0, t, 32, error</code>

操作

- 1. ARM64 操作 “IRQ 操作”操作
- 2. `EL1t` `EL1h` 操作 `SP_EL0` 操作 `SP_ELx` 操作

3. Linux `EL1h IRQ` `EL0 64-bit IRQ`

2.2 `kernel_ventry` `entry_handler`

`kernel_ventry`

1. `pt_regs`
2. `trampoline` `PAN/TTBR0`
3. `elX...label`

`entry_handler el, ht, regsize, label`

```
kernel_entry
-> x0 = sp (pt_regs)
-> bl el..._handler
-> ret_to_user / ret_to_kernel
```

ARM64

```
vectors
-> kernel_ventry
-> entry_handler el1h_64_irq / el0t_64_irq ...
-> C handler el1h_64_irq_handler
```

GICv3 `entry.S` `entry-common.c`

2.3

```
ARM64
-> el1h_64_irq_handler() arch/arm64/kernel/entry-
common.c
-> el1_interrupt()
-> handle_arch_irq set_handle_irq()
-> gic_handle_irq() drivers/irqchip/irq-gic-v3.c
-> gic_read_iar()
-> gic_complete_ack()
-> generic_handle_domain_irq()
-> irq_resolve_mapping()
-> handle_irq_desc()
-> desc->handle_irq()
-> handle_fasteoi_irq() / handle_percpu_devid_irq()
-> action->handler()
```

2.4

1. `arch/arm64/kernel/entry-common.c` `el1h_64_irq_handler()` `el1_interrupt(regs, handle_arch_irq)`
2. `kernel/irq/handle.c` `set_handle_irq()` `handle_arch_irq` `GIC` `irq_chip`
3. `drivers/irqchip/irq-gic-v3.c` `gic_init_bases()` `set_handle_irq(gic_handle_irq)`
4. `kernel/irq/irqdesc.c` `generic_handle_domain_irq()` `irq_resolve_mapping(domain, hwirq)` `desc` `flow handler`

2.5 NMI/Pseudo-NMI

GICv3 `ARM64` `pseudo-NMI`

1. `supports_pseudo_nmis`
2. `gic_rpr_is_nmi_prio()`
3. `__gic_handle_irq_from_irqsoff()`
4. `generic_handle_domain_nmi()`

GIC CPU interface **PMR/RPR** **NMI** `gic_prio_init()`

2.6 Pseudo-NMI

Pseudo-NMI `ARM64` **Group1 interrupt** **NMI-like**

2.6.1

`CONFIG_ARM64_PSEUDO_NMI`

1. `CONFIG_ARM64_PSEUDO_NMI`
2. `irqchip.gicv3_pseudo_nmi=1`
3. `GICv3`
4. `GIC` `nmi_support_forbidden`

`irqchip.gicv3_pseudo_nmi`

1. `arch/arm64/Kconfig` `config ARM64_PSEUDO_NMI` `ARM GICv3`
2. `Documentation/admin-guide/kernel-parameters.txt` `irqchip.gicv3_pseudo_nmi`
3. `drivers/irqchip/irq-gic-v3.c` `gic_enable_nmi_support()` `gic_prio_masking_enabled()` `nmi_support_forbidden`

2.6.2

`dist_prio_nmi`

1. `dist_prio_nmi = GICV3_PRIO_NMI`
2. `ICC_PMR_EL1` `CPU` `IRQ`
3. `ICC_RPR_EL1` `GICV3_PRIO_NMI` `NMI`

pseudo-NMI interrupt INTID **INTID** + **PMR/RPR**

2.6.3 ARM64

arch/arm64/kernel/entry.S CONFIG_ARM64_PSEUDO_NMI ICC_PMR_EL1

1. ICC_PMR_EL1
2. GIC_PRIO_IRQON | GIC_PRIO_PSR_I_SET
3. ICC_PMR_EL1
4. relaxed sync redistributor

pseudo-NMI GIC ARM64

2.6.4 GIC

drivers/irqchip/irq-gic-v3.c

```
gic_enable_nmi_support()
-> static_branch_enable(&supports_pseudo_nmis)
-> gic_{eoimodel_,}chip.flags |= IRQCHIP_SUPPORTS_NMI

irq_set_nmi()
-> gic_irq_nmi_setup()
-> desc->handle_irq
-> dist_prio_nmi

-> gic_handle_irq()
-> gic_read_iar()
-> gic_rpr_is_nmi_prio()
-> __gic_handle_nmi()
-> generic_handle_domain_nmi()
```

1. gic_enable_nmi_support() pseudo-NMI
2. gic_irq_nmi_setup() IRQ NMI flow handler
3. gic_irq_nmi_tearardown() NMI
4. gic_rpr_is_nmi_prio() ICC_RPR_EL1 NMI
5. __gic_handle_irq_from_irqsoff() IRQ pseudo-NMI

2.6.5 flow handler

IRQ pseudo-NMI irq_desc->handle_irq

1. SPI/ESPI handle_fasteoi_nmi
2. PPI/EPPI/SGI redistributor handle_percpu_devid_fasteoi_nmi

teardown

1. `handle_fasteoi_irq`
2. `handle_percpu_devid_irq`

pseudo-NMI “” **Linux IRQ core** **NMI-safe flow handler**

2.6.6 `gic_prio_init()`

`gic_prio_init()` pseudo-NMI

1. `GICD_CTLR.DS`
2. `SCR_EL3.FIQ`
3. Distributor priority `ICC_PMR_EL1/ICC_RPR_EL1`

1. NMI
2. CPU interface
3. `gic_rpr_is_nmi_prio()`

pseudo-NMI “” **GICD/GICR** **ICC sysreg**

2.6.7

1. LPI `gic_irq_nmi_setup()` `hwirq >= 8192`
2. enable IRQ NMI
3. pseudo-NMI
4. pseudo-NMI “”

2.6.8 pseudo-NMI

1. `arch/arm64/Kconfig` `CONFIG_ARM64_PSEUDO_NMI`
2. `arch/arm64/kernel/entry.S` `ICC_PMR_EL1`
3. `arch/arm64/kernel/entry-common.c` `__el1_pnmi()` `el1_interrupt()`
4. `kernel/entry/common.c` `irqentry_nmi_enter()` / `irqentry_nmi_exit()`
5. `drivers/irqchip/irq-gic-v3.c` `gic_prio_init()` `gic_enable_nmi_support()` `gic_irq_nmi_setup()` `gic_rpr_is_nmi_prio()` `__gic_handle_irq_from_irqsoff()`

2.6.9 pseudo-NMI

ARM64 GICv3 pseudo-NMI **ARM64** **Linux IRQ core** **NMI-like**

2.7 Linux Hardirq / Softirq / Tasklet / Threaded IRQ

GICv3 top half “ack/eoi” Linux

2.7.1 Hardirq

□ GICv3 □□□hardirq □□□□□

```

□□□□
-> irq_enter_rcu()
-> handle_arch_irq
-> gic_handle_irq
-> generic_handle_domain_irq
-> handle_fasteoi_irq / handle_percpu_devid_irq
-> primary handler
-> irq_exit_rcu()

```

hardirq □□□□

1. □□□□□□□□□□
2. □□□□□
3. □□□□□□□□□□
4. □□□□□□□□□□ softirq □□□□

□ kernel/softirq.c □□irq_exit_rcu() □□□□ hardirq □□□ local_softirq_pending()□□□□□□□□
invoke_softirq()□

2.7.2 Softirq□□□□□□□□□ CPU □□□□

softirq □□□□□□ **per-CPU pending bit + □□ action** □□

1. `softirq_vec[]` □□□□□ softirq □□□□□□□
2. `raise_softirq_irqoff()` □□ pending bit□
3. `__do_softirq()` □□ pending bit□□□□□□ action□
4. □□□□□□□□□□□□□□□ `ksoftirqd/<cpu>` □□□□□□

□ kernel/softirq.c □□□□□□□□□□□□□□

1. □ CPU □□□□□□□□□□□
2. □□□□□□□□□□□□□□□□
3. □□□□□ softirq □□□□□□□□□□□□□□

□□□□□□□□□□ softirq □□□□□□

1. HI
2. TIMER
3. NET_TX
4. NET_RX
5. BLOCK
6. IRQ_POLL
7. TASKLET
8. SCHED
9. HRTIMER
10. RCU

2.7.3 Tasklet `softirq` `tasklet`

`tasklet` `softirq` `tasklet` `softirq`

1. `__tasklet_schedule()` `tasklet` `per-CPU`
2. `TASKLET_SOFTIRQ` `HI_SOFTIRQ`
3. `tasklet_action_common()` `softirq` `tasklet`

`tasklet`

1. `tasklet`
2. `softirq`
3. “bottom half”

`kernel/workqueue.c`

TODO: Convert all tasklet users to workqueue and use softirq directly.

`tasklet`

2.7.4 Threaded IRQ

`request_threaded_irq()` `Linux`

1. primary handler `hardirq`
2. “”
3. `IRQ_WAKE_THREAD`
4. `irq thread` `thread_fn`

`kernel/irq/manage.c`

1. `request_threaded_irq()` `irqaction`
2. `handler == NULL` `primary handler` `IRQ_WAKE_THREAD`
3. `irq_thread_fn()` `thread_fn`
4. `IRQF_ONESHOT`

`hardirq`

2.7.5

```
Hardirq
->
-> raise softirq

Softirq
-> CPU
->
```

Tasklet

- > `softirq`
- > `tasklet`

Threaded IRQ

- > `hardirq`
- > `threaded IRQ`

GICv3 `hardirq` `threaded IRQ` `softirq` `tasklet`

2.8

GICv3 “bug”

2.8.1

1. `hardirq`
2. `softirq/tasklet`
3. `threaded irq`

`mutex` `synchronize_irq()` I/O

2.8.2 `local_irq_disable()`

`local_irq_disable()` CPU

1. CPU
2. CPU handler
3. NMI/pseudo-NMI

1. `per-CPU`
2. `spin_lock_irqsave()` IRQ

2.8.3 `local_bh_disable()`

`local_bh_disable()` CPU `softirq/bottom half`

1. `softirq`
2. `bottom half`
3. `spin_lock_irqsave()`

`local_irq_disable()`

2.8.4 `spin_lock_irqsave()` / `raw_spin_lock_irqsave()`

⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️

⚠️⚠️⚠️

1. `spin_lock_irqsave()` ⚠️⚠️⚠️⚠️⚠️/CPU ⚠️⚠️⚠️⚠️
2. `raw_spin_lock_irqsave()` ⚠️⚠️⚠️⚠️⚠️⚠️ irq core ⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️

⚠️ IRQ ⚠️⚠️ GIC ⚠️⚠️⚠️⚠️⚠️ `raw_spin_lock_irqsave()` ⚠️⚠️⚠️⚠️⚠️

1. ⚠️⚠️⚠️
2. ⚠️⚠️⚠️⚠️⚠️⚠️
3. ⚠️⚠️⚠️ hardirq/NMI ⚠️⚠️⚠️

2.8.5 `disable_irq_nosync()` / `disable_irq()` / `synchronize_irq()` ⚠️⚠️

⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️

1. `disable_irq_nosync(irq)` ⚠️⚠️⚠️⚠️⚠️⚠️⚠️ handler ⚠️⚠️
2. `disable_irq(irq)` ⚠️⚠️⚠️⚠️ `synchronize_irq()` ⚠️⚠️ handler ⚠️⚠️⚠️
3. `synchronize_irq(irq)` ⚠️⚠️⚠️ enable ⚠️⚠️⚠️⚠️⚠️ IRQ ⚠️ hardirq ⚠️ threaded handler ⚠️⚠️

⚠️ `kernel/irq/manage.c` ⚠️⚠️⚠️⚠️

1. `__synchronize_irq()` ⚠️⚠️ hardirq handler ⚠️⚠️
2. ⚠️⚠️ `threads_active` ⚠️ 0

⚠️⚠️⚠️ API ⚠️⚠️⚠️⚠️⚠️

1. ⚠️⚠️⚠️⚠️⚠️⚠️⚠️ handler ⚠️⚠️ `disable_irq_nosync()` /
2. ⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️ `disable_irq()` / `disable_irq_nosync()` + `synchronize_irq()` /
3. ⚠️⚠️ handler ⚠️⚠️⚠️⚠️⚠️ `synchronize_irq()` ⚠️⚠️⚠️⚠️⚠️

2.8.6 `synchronize_hardirq()` / `synchronize_irq()` ⚠️⚠️

⚠️⚠️⚠️⚠️⚠️ irq thread⚠️

1. `synchronize_hardirq()` ⚠️⚠️⚠️⚠️⚠️⚠️
2. `synchronize_irq()` ⚠️⚠️⚠️ threaded handler ⚠️⚠️

⚠️ threaded IRQ ⚠️⚠️⚠️⚠️⚠️

2.8.7 `IRQF_ONESHOT` ⚠️⚠️⚠️

⚠️⚠️ `IRQF_ONESHOT` ⚠️“⚠️⚠️⚠️⚠️⚠️⚠️ flag”⚠️⚠️⚠️⚠️⚠️

⚠️⚠️⚠️⚠️⚠️

1. hardirq primary handler ⚠️ thread ⚠️
2. ⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️⚠️

`irq_finalize_oneshot()` の呼び出し

2.8.8 GICv3 の動作

1. “GICv3 の動作”の CPU
2. “disable_irq() の動作”の in-flight handler と threaded handler
3. “tasklet と softirq”の動作
4. “threaded irq の動作”の primary handler と mask/unmask の動作

2.8.9 動作

動作のフロー

1. GICv3 の動作
2. IRQ と hardirq の動作
3. softirq/tasklet の動作
4. “GICv3 の動作”の “handler の動作”

動作の API の呼び出し

3 GICv3 の動作

3.1 Device Tree/ACPI の動作

動作のフロー

1. `gic_of_init()`
2. `gic_acpi_init()`

動作の API

```
gic_init_bases()
-> irq_domain_create_tree()
-> set_handle_irq(gic_handle_irq)
-> gic_update_rdist_properties()
-> gic_cpu_sys_reg_enable()
-> gic_prio_init()
-> gic_dist_init()
-> gic_cpu_init()
-> gic_enable_nmi_support()
-> gic_smp_init()
-> gic_cpu_pm_init()
-> its_init() / gicv2m_init()
```

3.2 Distributor の動作 `gic_dist_init()`

動作のフロー

1. Distributor

2. SPI/ESPI Non-secure Group-1
- 3.
- 4.
5. ARE_NS Group1
6. GICD_IROUTER SPI boot CPU

1. ARE affinity CPU mask
2. SPI/ESPI GICD SGI/PPI/EPPI GICR

3.3 Redistributor gic_populate_rdist() + gic_cpu_init()

GICv3 per-CPU “CPU GIC”

1. gic_iterate_rdist() redist region
2. __gic_populate_rdist() GICR_TYPER MPIDR affinity CPU
3. CPU rd_base phys_base
4. gic_enable_redist(true) redistributor
5. SGI/PPI Group1
6. gic_cpu_sys_reg_init() ICC system registers

3.4 CPU Interface gic_cpu_sys_reg_init()

GICv3

1. ICC_PMR_EL1
2. ICC_BPR1_EL1
3. ICC_CTLR_EL1 EOI RSS
4. ICC_AP0R* / ICC_AP1R*
5. ICC_IGRPEN1_EL1 Group1

CPU GICv3

4

4.1 GICv3

```
gic_chip_data
|- dist_base / dist_phys_base
|- redist_regions[]
|- rdists
|   |- per-cpu rdist
|   |   |- rd_lock
|   |   |- rd_base
|   |   |- pend_page
|   |   |- phys_base
|   |- prop_table_pa / prop_table_va
```

```

|   |- gicd_typer / gicd_typer2
|   |- has_vlpiis / has_rvpeid / has_direct_lpi
|- domain
|- flags / has_rss / ppi_nr

```

4.2 struct gic_chip_data

struct **GIC** struct gic_chip_data

struct

1. **dist_base** GICD struct
2. **redist_regions** GICR struct
3. **rdists** per-CPU redistributor LPI struct
4. **domain** GIC struct irq_domain
5. **has_rss** RSS
6. **ppi_descs** partitioned PPI struct

4.3 struct rdists

include/linux/irqchip/arm-gic-v3.h

GICv3 LPI/vPE struct “per-CPU struct”

1. **rdist** CPU redistributor struct
2. **prop_table_pa/va** LPI property table
3. **has_vlpiis** LPI
4. **has_rvpeid** RVPEID
5. **has_direct_lpi** direct LPI invalidation

4.4 ITS struct

ITS struct irq-gic-v3-its.c

1. **its_node** ITS struct BASER collection struct
2. **its_device** ITS DeviceID struct ITT event->LPI struct
3. **its_collection** redistributor/CPU struct
4. **event_lpi_map** LPI event id struct
5. **its_cmd_desc** Linux struct
6. **its_cmd_block** 4x64-bit struct

4.5 struct

```

struct ITS struct its_node
-> struct its_device
-> struct its_device struct EventID
-> EventID struct LPI
-> LPI struct collection struct CPU/redistributor
-> struct cmd_queue struct ITS struct MAPD/MAPTI/MOVI/INV/INT/CLEAR

```


5. 5.1 INTID 范围

INTID 范围

5.1 INTID 范围 `__get_intid_range()`

INTID

1. INTID 范围
2. $O(1)$ 复杂度
3. 与 flow handler 无关 affinity 无关

INTID GICv3 范围

5.2 转换 `convert_offset_index()`

转换

1. EPPI 范围 PPI 范围 index
2. ESPI 范围 `GICD_*nE` 范围 offset 范围

转换“范围”范围

5.3 ACK/EOI/Deactivate

ACK/EOI/Deactivate

```
gic_handle_irq()
-> gic_read_iar()
-> gic_complete_ack()
-> generic_handle_domain_irq()
-> handle_fasteoi_irq() / handle_percpu_devid_irq()
-> gic_eoi_irq()  gic_eoimodel_eoi_irq()
```

ACK/EOI/Deactivate

1. `gic_complete_ack()` EOI mode 1 范围 `ICC_E0IR1_EL1` 范围 priority drop
2. 范围 deactivate 范围 `gic_eoimodel_eoi_irq()` 范围 `ICC_DIR_EL1` 范围 `GICD_ICACTIVER`
3. 范围 forwarded-to-vcpu 范围 deactivate 范围

5.4 设置 `gic_set_affinity()`

SPI/ESPI 范围 affinity 范围

1. 范围 CPU
2. 范围 mask
3. 范围 `GICD_IROUTER` 范围
4. 范围 `gic_cpu_to_affinity(cpu)` 范围 Aff3:Aff0 范围

5. 寄存器 unmask

寄存器 $\$O(1)$ 的 `enable` 位 `RWP` 位

5.5 SGI 函数 `gic_compute_target_list()`

该 IPI 寄存器

该

1. 该 CPU 的 cluster 号
2. 该 cluster 的 CPU 的 16-bit target list
3. 该 cluster 的 `ICC_SGI1R_EL1`

该 **GICv3** 的 **SGI** 寄存器“该 CPU 的”“该 **affinity cluster** 的”

5.6 ITS LPI 分配器

ITS 的 LPI allocator

该

1. `lpi_range_list` 的 LPI 范围
2. `struct lpi_range { base_id, span }`

该

1. `alloc_lpi_range()` 的 **first-fit**
2. `free_lpi_range()` 的 base 范围

该

1. 该 $\$O(n)$
2. 该 $\$O(n)$

该“该” MSI/LPI 寄存器

5.7 ITS 寄存器

ITS 的 ring buffer

1. `its_allocate_entry()` 的 `CWRITER/CREADR` 寄存器
2. `its_post_commands()` 的 `GITS_CWRITER`
3. `its_wait_for_range_completion()` 的 `GITS_CREADR` 寄存器

该“该”寄存器

1. 该
2. 该
3. 该
4. 该 **SYNC/VSNC** 寄存器

5.8 ITS 函数 `its_select_cpu()`

ITS の LPI 間の CPU 間での割り当て

1. 各 NUMA node
2. 各 online mask と affinity mask
3. 各 `cpumask_pick_least_loaded()` による CPU

割り当て

6 GICv3 の IRQ domain

6.1 GIC domain

`gic_irq_domain_ops` の定義

1. `translate`
2. `alloc`
3. `free`
4. `select`

6.2 `gic_irq_domain_translate()` の実装

各 hwirq の

1. Device Tree SPI: `param[0] == 0` → `hwirq = param[1] + 32`
2. Device Tree PPI: `param[0] == 1` → `hwirq = param[1] + 16`
3. ESPI / EPPI: `ESPI_BASE_INTID / EPPI_BASE_INTID`
4. LPI: `hwirq`
5. SGI: `1 < hwirq < 16`

各 IRQ domain の “名前” の定義

6.3 `gic_irq_domain_map()` の hwirq 間の flow handler

各

1. SGI/PPI/EPPI → `handle_percpu_devid_irq`
2. SPI/ESPI → `handle_fasteoi_irq`
3. LPI → `handle_fasteoi_irq`

各

1. `irq_chip` → `gic_chip` → `gic_eoimodel_chip`
2. `irq_set_percpu_devid()`
3. `irqd_set_single_target()`
4. `irqd_set_handle_enforce_irqctx()`

各 の flow handler → **Linux flow handler** の実装

7 ITS/LPI/MSI の実装

7.1 中断分发

```
MSI 分发
-> ITS_TRANSLATER
-> DeviceID + EventID
-> ITT 索引
-> 中断 LPI
-> LPI 索引 collection 索引 redistributor
-> CPU 索引
```

GICv3 的 ITS 中断分发过程“中断分发器”

7.2 ITS 初始化

函数 `its_init()`

1. 通过 DT/ACPI 获取 ITS 信息
2. `its_probe_one()` 探测 ITS
3. 初始化 CBASER
4. 通过 BASER 获取 Device table、Collection table、VPE table
5. 获取 MSI domain
6. `allocate_lpi_tables()` 分配 PROPBASE/PENDBASE
7. CPU online 时 `its_cpu_init()`、`its_cpu_init_lpis()`、`its_cpu_init_collections()`

7.3 LPI 索引

1. **PROPBASE** 索引 LPI 索引表 Group1 索引
2. **PENDBASE** 索引 pending bitmap

函数

1. `its_setup_lpi_prop_table()`
2. `its_cpu_init_lpis()`
3. `lpi_write_config()`
4. `lpi_update_config()`

7.4 MSI domain 索引

ITS 索引 IRQ 索引 GIC parent domain 索引 MSI domain

1. GIC 索引 domain 索引 hwirq
2. ITS parent MSI domain 索引
3. `its_irq_chip` 索引 mask/unmask/eoi/compose_msi_msg/set_affinity 索引

ITS 索引 **hierarchical irq_domain**

7.5 中断 ITS 索引

1. **MAPD** 索引 DeviceID 索引 ITT
2. **MAPC** 索引 collection 索引 redistributor

3. **MAPTI** 设备ID/EventID 映射到 INTID/LPI
4. **MOVI** 创建 collection
5. **INV** 事件 注册
6. **INVAL** 删除 collection
7. **INT** pending
8. **CLEAR** pending
9. **DISCARD** 删除

函数 `its_build_*_cmd()` 在 `linux` 中定义

8 GICv4 架构

8.1 GICv4 概述

“GICv4”架构

1. 在 guest 中运行 GIC
2. 在 hypervisor 中运行

8.2 组件

1. **vLPI** guest 中的 LPI
2. **vPE** virtual Processing Element 在 guest vCPU 中 GIC
3. **doorbell** vPE 中的 doorbell LPI 在 hypervisor 中

8.3 操作

1. `drivers/irqchip/irq-gic-v4.c` 在 hypervisor 中
2. `irq_set_vcpu_affinity()` 在 `ioctl` 中
3. `its_vlpi_map()` / `its_vlpi_unmap()` / `its_prop_update_vlpi()` 在 LPI 中 LPI
4. `its_make_vpe_resident()` / `its_make_vpe_non_resident()` / `its_commit_vpe()` 在 vPE 中

操作

1. 在 GICv3 + ITS 中
2. 在 GICv4 中 host 在 guest 中

9 实现

9.1 实现

实现

1. `drivers/irqchip/irq-gic-v3.c`
2. `drivers/irqchip/irq-gic-v3-its.c`
3. `arch/arm64/kernel/entry-common.c`
4. `kernel/irq/irqdesc.c`
5. `kernel/irq/chip.c`

6. `include/linux/irq.h`
7. `include/linux/irqdesc.h`
8. `include/linux/irqdomain.h`
9. `include/linux/irqchip/arm-gic-v3.h`

□□□□□□□□

1. `drivers/irqchip/irq-gic-v4.c`
2. `drivers/irqchip/irq-gic-v3-mbi.c`
3. `arch/arm64/include/asm/irq.h`
4. `drivers/irqchip/irq-gic-common.h`

9.2 □□□□□□

1. `include/linux/irq.h`
2. `include/linux/irqdesc.h`
3. `include/linux/irqdomain.h`
4. `kernel/irq/chip.c`
5. `kernel/irq/irqdesc.c`
6. `arch/arm64/kernel/entry-common.c`
7. `drivers/irqchip/irq-gic-v3.c`
8. `drivers/irqchip/irq-gic-v3-its.c`
9. `drivers/irqchip/irq-gic-v4.c`
10. `drivers/irqchip/irq-gic-v3-mbi.c`

□□□□“□□□ `irq-gic-v3.c`”□□□□□

□10□□□□□□□□□□□□□□□□

□□A□□□□□□□□□□□□□1□□

□□□□□□□□□□□□ GICv3 □□□□□□

□□□

1. □□ `irq_desc`□`irq_data`□`irq_chip`□`irq_domain`□
2. □□□□ “`hwirq -> irq_domain -> virq -> irq_desc -> action`” □□
3. □□ `irq-gic-v3.c` □□ `gic_init_bases`□`gic_irq_domain_translate`□
`gic_irq_domain_map`□`gic_handle_irq`□`gic_set_affinity`□`gic_ipi_send_mask`□
4. □□ `arm64-reference/gic` □□□□□□□□□□□□GICD□GICR□ICC□GITS□

□□□

1. □□□□□□□□□□□□
2. □□□□□□□□□□□□□□
3. □□□□□□□□□□

□□B□□□□□□□□□□□□□□2□□

□□□□□□□□□□□□ IRQ □□□□□□□□□□ ISR□

1. `cat /proc/interrupts | grep -i gic`
2. `cat /proc/interrupts | grep -i gic`
3. `cat /proc/interrupts`
4. `cat /proc/interrupts | grep -i gic`
5. `cat /proc/interrupts`

1. PPI → `handle_percpu_devid_irq`
2. SPI → `handle_fasteoi_irq`
3. EOI mode 1 → priority drop → deactivate →

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

1. `ioctl SPI & affinity && /proc/irq/N/smp_affinity_list`
2. `gic_set_affinity()`
3. `gic_ipi_send_mask()` `ICC_SGI1R_EL1`
4. `RSS` `cluster`

1. `gic_compute_target_list()` 计算 cluster 的 target
2. `gic_cpu_to_affinity()` 根据 MPIDR 计算 Aff3:Aff0
3. `GICD IROUTER` 与 `ICC SGI1R EL1` 交互

MSI LPI delivery

1. 创建 ITS
2. `its_init()` `its_probe_one()` `its_cpu_init_lpis()` `its_irq_chip`
3. `MAPD/MAPC/MAPTI/MOVI/INV` 寄存器
4. 驱动 `irq-gic-v4.c`

1. PROPBASE → PENDBASE □□□□□□□□
2. □□□ ITS □□□□□□□□ flush / sync□
3. □□□ GICv4 □□□ vPE □ doorbell□

11.1

launch.sh ARM64 QEMU GICv3 QEMU GICv3

1. QEMU gic-version=3
2. dmesg GICv3
3. ITS QEMU ITS

11.2

1 GICv3

```
dmesg | grep -i gic
cat /proc/interrupts
```

1. GICv3 features
2. Pseudo-NMIs enabled
3. ITS: Enabling GICv4 support
4. CPU redistributor

2 IRQ

```
echo function_graph > /sys/kernel/tracing/current_tracer
echo gic_handle_irq > /sys/kernel/tracing/set_graph_function
echo generic_handle_domain_irq >> /sys/kernel/tracing/set_graph_function
echo handle_fasteoi_irq >> /sys/kernel/tracing/set_graph_function
echo 1 > /sys/kernel/tracing/tracing_on
cat /sys/kernel/tracing/trace_pipe
```

function_graph trace event

```
echo 1 > /sys/kernel/tracing/events/irq/irq_handler_entry/enable
echo 1 > /sys/kernel/tracing/events/irq/irq_handler_exit/enable
cat /sys/kernel/tracing/trace_pipe
```

3 affinity

```
cat /proc/irq/<irq>/smp_affinity_list
echo 2 > /proc/irq/<irq>/smp_affinity_list
```


555

- 4 IPI**

```
workqueue TLB shutdown  smp_call_function
```

5 GDB

1. `elh_64_irq_handler`
2. `gic_handle_irq`
3. `generic_handle_domain_irq`
4. `handle_fasteoi_irq`
5. `irq_desc` ISR

 “ ” “ ”

11.3 ITS 分析

□□□□□ ITS □□□□□□□□□□

1. 初始化 ITS probe
2. 初始化 LPI property table 和 pending table
3. 初始化 MSI 和 LPI
4. 调用 `its_irq_compose_msi_msg()`, `its_set_affinity()`, `its_send_mapti()`, `its_send_inv()`

11.4 QEMU 虚拟机

```
##### "#####" ##### launch.sh ##### QEMU #####
```

□□□A□□□□ GICv3 □□□□□□

11

QEMU GICv3 gic of init() gic init bases()

11

launch.sh -machine virt -M virt,gic-version=3

```
qemu-system-aarch64 \
  -M virt,gic-version=3 \
  -cpu cortex-a57 \
  -smp 4 \
  -m 1024 \
  -kernel arch/arm64/boot/Image \
  --append "nokaslr rdinit=/linuxrc console=ttyAMA0" \
  -nographic \
  --fsdev local,id=kmod_dev,path=$PWD/kmodules,security_model=none \
  -device virtio-9p-device,fsdev=kmod_dev,mount_tag=kmod_mount
```

-M virt,gic-version=3

```
dmesg | grep -Ei "gic|its|redistributor|ppi|lpi"
cat /proc/interrupts
```

1. GICv3
2. CPU redistributor
3. GICv3 features Pseudo-NMIs enabled ITS
4. /proc/interrupts timer IPI

1. gic_of_init()
2. gic_init_bases()
3. gic_update_rdist_properties()
4. gic_dist_init()
5. gic_cpu_init()

A-1 Pseudo-NMI

QEMU ARM64 GICv3 pseudo-NMI

AAAAA

```
--append "nokaslr rdinit=/linuxrc console=ttyAMA0
irqchip.gicv3_pseudo_nmi=1"
```

AAAAAAAAAAAA

```
grep CONFIG_ARM64_PSEUDO_NMI .config
```

AAA

AAAAAA

```
dmesg | grep -Ei "Pseudo-NMI|PMR|GICv3"
```

AAAAAAAAAAAA

```
Pseudo-NMIs enabled using ... ICC_PMR_EL1 synchronisation
```

AAA `gic_enable_nmi_support()` AAAAAAA `supports_pseudo_nmis` AAA static key AAAA

AAAAAA

AAAA“AAAA”AAAAAAAAAAAAAAAAAAAAAAAAAAAA GDB AAA

```
b gic_rpr_is_nmi_prio
b __gic_handle_irq_from_irqsoff
b __gic_handle_nmi
b generic_handle_domain_nmi
```

AAAAAA

AAA QEMU `virt` AAAAAAAAAAAAAAA **pseudo-NMI flow handler** AAAAAAA

1. AAAAAAAAAAAAAAA NMI AAAA
2. QEMU CPU AAAAAAAAAAA PMU/AAAAAAAAAA
3. AAAAAAAAAAAAAAA IRQ AAAA NMI

AAAAAAAAAAAA

1. AAAAAAA pseudo-NMI AAAAAAAAAAA
2. AAAAAAA PMU AAAAAAAAAAAAAAA NMI handler AAA

1. `gic_enable_nmi_support()`
2. `gic_irq_nmi_setup()`
3. `gic_rpr_is_nmi_prio()`
4. `__gic_handle_irq_from_irqsoff()`
5. `irqentry_nmi_enter() / irqentry_nmi_exit()`

1. 8 pseudo-NMI
2. 16 pseudo-NMI
3. 32 pseudo-NMI

1. GIC 寄存器
2. per-cpu 寄存器
3. `handle_percpu_devid_irq()` 寄存器

```
cat /proc/interrupts
cat /proc/interrupts | grep -Ei "arch|timer|cntv|phys"
```

```
echo 0 > /sys/kernel/tracing/tracing_on
echo nop > /sys/kernel/tracing/current_tracer
echo > /sys/kernel/tracing/trace
echo 1 > /sys/kernel/tracing/events/irq/irq_handler_entry/enable
echo 1 > /sys/kernel/tracing/events/irq/irq_handler_exit/enable
echo 1 > /sys/kernel/tracing/tracing_on
cat /sys/kernel/tracing/trace pipe
```

28 / 59

```
for i in 1 2 3 4; do
    sh -c 'while ;; do ;; done' &
done
```

□□□□□□

1. □□□□□□ CPU □□□□□□
2. timer □□□□□□□□ per-cpu □□□
3. trace □□□□□□ timer ISR □□□□□□
4. □□□□□□□□□□□□□□ `handle_percpu_devid_irq()`□

□□□□

1. `gic_irq_domain_map()` □ PPI □□□□□□
2. `handle_percpu_devid_irq()`□
3. `gic_cpu_init()` □ GICR SGI/PPI □□□□□□□□

□□□□

□□□“PPI □□ CPU □□□□”□□□□□□□□□□□□

□□□C□SPI □□□□□□□□□□□□□□□□ affinity □□

□□

□□□□□□□ `GICD + IROUTER + handle_fasteoi_irq` □□□□□□ affinity □□□□□□□□□□

□□□□□□

□□□□□□□□□□□□□□□□

1. □□□□□□□□□□ `virtio-9p-device`□
2. □□□□□□□□□□□□□□□□ `virtio-blk-device` □ `virtio-net-device`□□□□□□□□□□□□ I/O □□□

□□□□□ **SPI** □□□ **QEMU** □□□□

```
qemu-system-aarch64 \
-M virt,gic-version=3 \
-cpu cortex-a57 \
-smp 4 \
-m 1024 \
-kernel arch/arm64/boot/Image \
--append "nokaslr rdinit=/linuxrc console=ttyAMA0" \
-nographic \
--fsdev local,id=kmod_dev,path=$PWD/kmodules,security_model=none \
-device virtio-9p-device,fsdev=kmod_dev,mount_tag=kmod_mount \
```

```
-drive if=none,file=disk.img,format=raw,id=vdisk \
-device virtio-blk-device,drive=vdisk
```

disk.img を 9p で接続する

手順

1. 9p 接続の準備
2. 9p CPU 接続
3. affinity
4. I/O 接続

準備

```
cat /proc/interrupts
cat /proc/irq/<irq>/smp_affinity_list
echo 2 > /proc/irq/<irq>/smp_affinity_list
cat /proc/irq/<irq>/smp_affinity_list
watch -n 1 cat /proc/interrupts
```

9p 接続の準備

```
while true; do
    find / > /dev/null 2>&1
done
```

接続

1. CPU 接続
2. affinity を CPU 接続
3. gic_set_affinity() を GICD_IROUTER で実行

接続

1. gic_set_affinity()
2. gic_cpu_to_affinity()
3. handle_fasteoi_irq()
4. gic_eoimodel_eoi_irq()

接続

SPI を GICD CPU で IROUTER で実行

D/IPI/SGI の接続

3. `gic_send_sgi()`
4. `gic_smp_init()`

□□□□

□□□“IPI □ SGI □ Linux □□□□”□□□□□□□□□□

□□□E□ITS/LPI/MSI □□□□

□□

□□□□ QEMU □□ ITS□□□□□□□□□□ MSI -> ITS -> LPI -> collection -> redistributor □□□□□

□□□□

□□□□□□□□□□□□□□□□□□□□□□□□□□□□

1. `CONFIG_ARM_GIC_V3_ITS=y`
2. `CONFIG_PCI=y`
3. `CONFIG_PCI_HOST_GENERIC=y` □□□□□ PCI host □□
4. `CONFIG_VIRTIO_PCI=y` □□□□□□□□□□ PCI □□

QEMU □□□□

```
qemu-system-aarch64 \
  -M virt,gic-version=3,its=on \
  -cpu cortex-a57 \
  -smp 4 \
  -m 1024 \
  -kernel arch/arm64/boot/Image \
  --append "nokaslr rdinit=/linuxrc console=ttyAMA0" \
  -nographic \
  -netdev user,id=n1 \
  -device virtio-net-pci,netdev=n1 \
  -device virtio-rng-pci
```

□□□□□□□□□□□□ `virtio-net-pci` □□ `virtio-blk-pci`□□□□□□□□□□□□□□□□□□□□□□□□□□□□ **PCI MSI/MSI-X**□□□□□
ITS/LPI □□□□□□□□

□□□□□□□□

```
dmesg | grep -Ei "ITS|LPI|GICv4|VPE|redistributor"
cat /proc/interrupts
```

□□□

1. 通过 ITS probe 初始化
2. 初始化 property table / pending table 结构
3. 初始化 8192 个 LPI 的 MSI/LPI 结构

初始化

1. `virtio-net-pci` 初始化
2. `virtio-blk-pci` 初始化
3. `virtio-rng-pci` 初始化

初始化

1. ITS 初始化
2. LPI property/pending table 初始化
3. 初始化 SPI 的 ITS/LPI 结构
4. `its_set_affinity()` 设置 LPI 的 collection 属性

初始化

1. `its_init()`
2. `its_probe_one()`
3. `its_cpu_init_lpis()`
4. `its_irq_chip`
5. `its_send_mapd()` / `its_send_mapti()` / `its_send_inv()` / `its_send_movi()`

初始化

通过“MSI 的 GICv3 初始化 GICD 的 ITS 的 LPI”初始化

通过 F/GDB 的 flow handler

通过

“ARM64 的 GIC 的 IRQ core flow handler -> 的 ISR”初始化

初始化

初始化

```
./launch.sh arm64 debug
```

通过 QEMU 的 `-s -S` 初始化

1. GDB stub 的 `localhost:1234`
2. CPU 的初始化

通过初始化

gdb-multiarch

10 10 10 10 10 10 10 10 10 10

```
fix_vmlinux_head_entry.sh IRQ
_text
```

□ □ □ □ □ □ □ □ □

□□□□□□□□“□□ IRQ □□□□□□□□”□□□□□□□□□□□□□□□□□□□□

□ □ □ □ □ □ □ □ □ □

1. `el1h_64_irq_handler` □ EL1 IRQ □□□□□ C handler□
2. `el1_interrupt` □ pseudo-NMI □□□□□ `__el1_pnmi()` □□□□ `__el1_irq()` □
3. `gic_handle_irq` □ ARM64 □ IRQ handler □□□□□□□ GICv3□
4. `__gic_handle_irq_from_irqson` □□□ IRQ □□□□□ IAR□□□ NMI □□□□□□□
5. `__gic_handle_irq_from_irqsoff` □ IRQ-off □□□□□□□□□ pseudo-NMI □□□□□□
6. `__gic_handle_irq` □ `irqnr` □ ACK □□□□□ domain □□
7. `generic_handle_domain_irq` □ `hwirq -> virq/desc` □□□□□□□
8. `handle_fasteoi_irq` □□□ GICv3 □ SPI/shared □□□□□□
9. `handle_percpu_devid_irq` □□□ timer □ per-CPU PPI □□□

10. `__handle_irq_event_percpu` の呼び出しと ISR

呼び出しの順序

呼び出しの `si` の値

1. 呼び出し `ni/si` の 3 の 10 進法で表した値を C
2. の C の値を `n`
3. の値を `s`
4. の `timer` の値を `tb` の値

呼び出しの `el1h_64_irq_handler` の値

```
delete breakpoints
tb el1_interrupt
tb gic_handle_irq
tb __gic_handle_irq_from_irqson
tb __gic_handle_irq
tb generic_handle_domain_irq
tb handle_fasteoi_irq
tb handle_percpu_devid_irq
tb __handle_irq_event_percpu
c
```

この GDB のコマンドで `timer` の値を確認

呼び出しの順序

の `el1h_64_irq_handler`

```
p/x regs->pc
p/x regs->pstate
```

呼び出し

1. 呼び出し EL1 IRQ の値
2. `regs` の値を `IRQ` の値

の `el1_interrupt`

```
p regs_irqs_disabled(regs)
p handler
```

呼び出し

1. 呼び出し IRQ-on の IRQ-off の値

2. handler `handle_arch_irq`

`__gic_handle_irq_from_irqson`

```
n
p/x irqnr
p is_nmi
```

動作

1. `irqnr = gic_read_iar()` から INTID を取得
2. `is_nmi` が偽の場合は pseudo-NMI

`__gic_handle_irq`

```
p/x irqnr
n
```

動作

1. special INTID (1020-1023) の場合
2. 通常の INTID の場合は `generic_handle_domain_irq()`

`generic_handle_domain_irq`

```
p/x hwirq
n
```

動作: API として **GIC** から **Linux IRQ domain** へ渡す

`handle_fasteoi_irq`

```
p desc->irq_data.irq
p/x desc->istate
p desc->action->name
```

動作

1. Linux virq から GIC hwirq を取得
2. fasteoi flow handler を呼び出す
3. `desc->action->name` が `fasteoi_irq` の場合

`handle_percpu_devid_irq`

```
p desc->irq_data.irq
p desc->action->name
```

このコードは、PPI/per-CPU の timer を

__handle_irq_event_percpu

```
p irq
p action->name
p action->handler
```

このコードは、ISR の s を action->handler() に渡す

このコードは、PPI/timer を

このコードは、timer PPI を

このコードは

```
delete breakpoints
tb ellh_64_irq_handler
tb ell_interrupt
tb gic_handle_irq
tb __gic_handle_irq_from_irqson
tb __gic_handle_irq
tb generic_handle_domain_irq
tb handle_percpu_devid_irq
tb __handle_irq_event_percpu
c
```

このコードは

1. handle_percpu_devid_irq() handle_fasteoi_irq()
2. action->name timer/percpu
3. このコードは“PPI CPU”

このコードは SPI/shared

このコードは QEMU の virtio-blk virtio-net SPI

```
delete breakpoints
tb gic_handle_irq
tb __gic_handle_irq_from_irqson
tb __gic_handle_irq
tb generic_handle_domain_irq
tb handle_fasteoi_irq
```

```
tb __handle_irq_event_percpu
c
```

guest I/O

1. `handle_fasteoi_irq()`
2. `desc->action->name` virtio handler
3. GICv3 “ + fasteoi”

pseudo-NMI

pseudo-NMI

1. `CONFIG_ARM64_PSEUDO_NMI`
2. GICv3 pseudo-NMI

```
delete breakpoints
tb ell_interrupt
tb __gic_handle_irq_from_irqsoff
tb __gic_handle_nmi
tb handle_fasteoi_nmi
c
```

IRQ pseudo-NMI

1. IRQ-off
2. `__gic_handle_irq_from_irqsoff()` PMR IAR
3. NMI-safe flow handler `handle_fasteoi_irq()`

1. `pt_regs` GIC hwirq Linux virq
2. PPI SPI pseudo-NMI
3. `handle_percpu_devid_irq()` `handle_fasteoi_irq()`
4. `desc->action->handler`
- 5.

GDB IRQ “”

“ GDB”

1. `/proc/interrupts` を確認し IRQ を確認
2. 確認可能な tracepoint を PPI/SPI の softirq を確認
3. GDB を確認可能な範囲で確認

確認可能な範囲で“確認”可能な範囲で確認可能な範囲で

確認

確認可能な範囲で確認可能な範囲で

1. 確認 IRQ を確認可能な範囲で ISR
2. 確認可能な範囲で PPI の SPI を確認 pseudo-NMI
3. “確認可能な範囲”を確認可能な範囲で GIC の IRQ core を確認可能な範囲で

11.5 確認可能な範囲

確認可能な範囲で確認可能な範囲で確認可能な範囲で

1. 確認 A を確認 GICv3 を確認可能な範囲で
2. 確認 B を PPI の CPU を確認
3. 確認 D を SGI/IPI
4. 確認 C を SPI の affinity
5. 確認 E を ITS/LPI/MSI
6. 確認 F を GDB を確認可能な範囲で

確認可能な範囲で確認可能な範囲で確認可能な範囲で確認可能な範囲で

12 確認可能な範囲を確認可能な範囲

確認可能な範囲で確認可能な範囲で“GICv3 を確認”を確認可能な範囲で確認可能な範囲で

12.1 確認可能な範囲で IRQ core を確認可能な範囲

確認可能な範囲で確認可能な範囲で

1. 確認
2. CPU 100%
3. `/proc/interrupts` を確認
4. IRQ を確認 handler を確認

確認可能な範囲で確認可能な範囲で

1. 確認可能な範囲で確認可能な範囲で
2. GIC を確認可能な範囲で hwirq を virq を確認 irq_domain
3. irq_desc を flow handler を trigger type を確認 IRQ core / irq_chip
4. flow handler を確認 ISR を確認可能な範囲で確認可能な範囲で

確認可能な範囲で確認可能な範囲で entry.S irq-gic-v3.c kernel/irq/* を確認可能な範囲で

12.2 “確認可能な範囲で CPU を確認”

1. 1. 2. 3. 4. 5.

1. 1. 2. 3. 4. 5.
1. 2. 3. 4. 5.
1. 2. 3. 4. 5.
1. 2. 3. 4. 5.
1. 2. 3. 4. 5.

12.3 12.3

12.3 12.3

1. 2. 3. 4.

1. 2. 3. 4.
1. 2. 3. 4.
1. 2. 3. 4.
1. 2. 3. 4.

12.4 12.4

12.4 12.4

1. 2. 3. 4.

1. 2. 3. 4.
1. 2. 3. 4.
1. 2. 3. 4.
1. 2. 3. 4.

12.5 12.5

12.5 12.5

1. 2. 3. 4.

1. 2. 3. 4.
1. 2. 3. 4.
1. 2. 3. 4.
1. 2. 3. 4.

12.6 12.6

1. 2. 3. 4.

1. 2. 3. 4.
1. 2. 3. 4.
1. 2. 3. 4.
1. 2. 3. 4.

12.7 12.7

以下は bug の原因を調査するための手順

1. ハード IRQ の確認
2. ソフト IRQ/tasklet の確認
3. ハンドラーの同期化 `synchronize_irq()`
4. Threaded IRQ の one-shot の確認

以下は CPU のアーキテクチャに関する情報

13. アーキテクチャ

以下はアーキテクチャに関する情報

1. CPU
2. ソフト IRQ

13.1 CPU

13.1.1

1. `/proc/interrupts` の CPU
2. CPU の CPU
3. `/proc/irq/<irq>/smp_affinity_list`
4. IO/virtio

13.1.2

affinity

1. SGI/PPI/EPPI per-CPU SPI
2. SPI/ESPI `gic_set_affinity()` `GICD_IROUTER`
3. LPI/MSI ITS collection

13.1.3

```
cat /proc/interrupts
cat /proc/irq/<irq>/smp_affinity_list
cat /proc/irq/<irq>/effective_affinity_list
ps -Leo pid,psr,comm,%cpu | egrep 'ksoftirqd|irq/'
cat /proc/softirqs
```

`effective_affinity_list` `/proc/interrupts`

13.1.4 “affinity”

affinity

1. IRQ affinity

2. 00/0000 managed IRQ000000 CPU 0000000000
3. 0000000000effective_affinity 000 mask 000
4. irqbalance 0000000000
5. 0000000000000000 IRQ000000 CPU 000000

13.1.5 0000000000

000 SPI 0 MSI/LPI000000000000

1. 000 IRQ 00000000 affinity0
2. 000000 irqbalance0000000000
3. 00 /proc/irq/<irq>/smp_affinity_list0
4. 00 /proc/interrupts 00000000
5. 00000000000000000000000000

00000000000000

1. 00 MSI-X queue 000
2. 00 queue 0 IRQ 0000000000
3. 00 RSS/RPS/XPS/RFS 000000 queue affinity 000
4. 000000000 queue/IRQ 0 NUMA00 CPU 00

000000000000“00 IRQ”000 0000000000000000000000000000000000

13.1.6 0000000000

000000

1. 00 CPU0 0000000000
2. 000 /proc/irq/N/smp_affinity_list 00 CPU20
3. 000000 CPU 0000000000

000000

1. 00000000 NET_RX softirq 000000 CPU 000
2. 000000000 CPU0
3. 00000000000000000000000000

000“0000000000000000”00000000

13.2 00000000softirq 00

13.2.1 0000

1. top/htop 00 ksoftirqd/N 000
2. /proc/softirqs 000000000000
3. 0000000000 CPU 00000000000000
4. 00000000000000000000 IO 000

13.2.2 0000000000 softirq

□□□□

- □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- ### 13.2.3 `hardirq` `softirq`

□□□□

--	--	--	--	--	--

- ### 13.2.4 ksoftirqd □□□□□□

□ □ □ □ □ □ □

1. `softirq` [] `irq_exit_rcu()` []
2. [] `ksoftirqd/<cpu>`

□□ ksoftirqd □□□□“□□□□□□□□”□□□ □□□□□□□□□□□□□□ □□□□

13.2.5 NET_RX softirq □□□□□□□□

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- ## 1. IRQ 00000000 CPU0

2. MSI-X
3. RSS/RPS/XPS/RFS
4. CPU
5. `netdev_budget`/`netdev_budget_usecs`

1. IRQ budget
2. CPU

13.2.6 TASKLET softirq

TASKLET

1. tasklet
2. tasklet softirq
3. threaded irq workqueue

tasklet workqueue/softirq

13.2.7 softirq trace

tracepoint

```
echo 1 > /sys/kernel/tracing/events/irq/softirq_raise/enable
echo 1 > /sys/kernel/tracing/events/irq/softirq_entry/enable
echo 1 > /sys/kernel/tracing/events/irq/softirq_exit/enable
cat /sys/kernel/tracing/trace_pipe
```

```
echo 1 > /sys/kernel/tracing/events/irq/irq_handler_entry/enable
echo 1 > /sys/kernel/tracing/events/irq/irq_handler_exit/enable
```

1. raise softirq
2. softirq
3. hardirq return `ksoftirqd`

13.3 softirq

1. CPU
2. softirq CPU

3. `virtio` `NAPI` `workqueue` `irq thread` `CPU`
4. `workqueue` `irq thread` `CPU`

Device/Queue

```
Device/Queue
-> IRQ CPU
-> Softirq CPU
-> Kernel worker / irq thread CPU
-> User thread CPU
```

Device/Queue `CPU`

13.4 `softirq` `irq`

“`softirq` `CPU`” “`softirq` `irq`”

```
cat /proc/interrupts
cat /proc/softirqs
ps -Leo pid,psr,comm,%cpu | egrep 'ksoftirqd|irq/'
cat /proc/irq/<irq>/smp_affinity_list
cat /proc/irq/<irq>/effective_affinity_list
```

Device/Queue

1. `PPI/SPI/LPI` `irq`
2. `hardirq` `softirq` `irq`
3. `irq/<n>-xxx` `ksoftirqd/N` `irq`
4. `GIC` `irq`

“`softirq` `CPU`”

13.5 `QEMU` “`softirq` `CPU`”

“`softirq` `CPU`” `affinity` `irq`

```
Device/Queue
-> GICD IROUTER / ITS collection
-> /proc/interrupts CPU
-> softirq /
```

13.5.1 `QEMU`

“`softirq` `CPU`” `QEMU` `I/O` `virtio-blk-device` `virtio-net-pci`

```
qemu-system-aarch64 \
-M virt,gic-version=3 \
-cpu cortex-a57 \
-smp 4 \
-m 1024 \
-kernel arch/arm64/boot/Image \
--append "nokaslr rdinit=/linuxrc console=ttyAMA0" \
-nographic \
--fsdev local,id=kmod_dev,path=$PWD/kmodules,security_model=None \
-device virtio-9p-device,fsdev=kmod_dev,mount_tag=kmod_mount \
-drive if=none,file=disk.img,format=raw,id=vdisk \
-device virtio-blk-device,drive=vdisk
```

13.5.2 □□□□

- □ □ □ □ □ □

□□ affinity□

□□□□□□□□□□ I/O □□□□

46 / 59

9p

```
while true; do
  find / -type f >/dev/null 2>&1
done
```

13.5.3

`smp_affinity_list`

1. `/proc/interrupts` IRQ CPU
2. `effective_affinity_list` CPU
3. `ksoftirqd/N` `irq/<n>-xxx` CPU

13.5.4 trace

```
echo 1 > /sys/kernel/tracing/events/irq/irq_handler_entry/enable
echo 1 > /sys/kernel/tracing/events/irq/irq_handler_exit/enable
echo 1 > /sys/kernel/tracing/events/irq/softirq_raise/enable
echo 1 > /sys/kernel/tracing/events/irq/softirq_entry/enable
echo 1 > /sys/kernel/tracing/events/irq/softirq_exit/enable
cat /sys/kernel/tracing/trace_pipe
```

1. IRQ handler CPU
2. `raise` `softirq`
3. `softirq` CPU

“CPU”“CPU”

13.6 QEMU “softirq”

`softirq` `NET_RX` `softirq`

13.6.1

QEMU `virtio-net-pci`

```
qemu-system-aarch64 \
  -M virt,gic-version=3 \
  -cpu cortex-a57 \
  -smp 4 \
  -m 1024 \
  -kernel arch/arm64/boot/Image \
```

```
--append "nokaslr rdinit=/linuxrc console=ttyAMA0" \
-nographic \
-netdev user,id=n1,hostfwd=tcp::10022-:22 \
-device virtio-net-pci,netdev=n1
```

guest の動作を確認する

1. TIMER の TASKLET softirq
2. block の softirq

“” NET_RX の動作を確認

13.6.2

確認

```
cat /proc/softirqs
ps -Leo pid,psr,comm,%cpu | egrep 'ksoftirqd|irq/'
```

trace

```
echo 1 > /sys/kernel/tracing/events/irq/softirq_raise/enable
echo 1 > /sys/kernel/tracing/events/irq/softirq_entry/enable
echo 1 > /sys/kernel/tracing/events/irq/softirq_exit/enable
echo 1 > /sys/kernel/tracing/events/irq/irq_handler_entry/enable
echo 1 > /sys/kernel/tracing/events/irq/irq_handler_exit/enable
cat /sys/kernel/tracing/trace_pipe
```

guest の I/O

- 1.
- 2.
- 3.

13.6.3

CPU

1. softirq_raise の IRQ
2. NET_RX の TIMER の TASKLET
3. softirq の irq_exit() の ksoftirqd/<cpu>
4. ksoftirqd/<cpu> の CPU

13.6.4

1. virtio IRQ CPU0
2. `NET_RX` softirq CPU0
3. `ksoftirqd/0`

“softirq”

1. IRQ
- 2.
3. NAPI / RSS / RPS

softirq

13.7

13.7.1 “CPU”

1. `drivers/irqchip/irq-gic-v3.c` `gic_set_affinity()` SPI/ESPI
2. `drivers/irqchip/irq-gic-v3.c` `gic_cpu_to_affinity()` CPU Aff3:Aff0
3. `drivers/irqchip/irq-gic-v3-its.c` `its_set_affinity()` `its_select_cpu()` LPI/MSI CPU MOVI
4. `kernel/irq/manage.c` `irq_set_affinity()` `irq_set_affinity_locked()` `irq_validate_effective_affinity()` IRQ core

13.7.2 “softirq”

1. `kernel/softirq.c` `raise_softirq_irqoff()` raise
2. `kernel/softirq.c` `__do_softirq()`
3. `kernel/softirq.c` `invoke_softirq()` `wakeup_softirqd()` `ksoftirqd`
4. `kernel/softirq.c` `tasklet_action_common()` tasklet
5. `kernel/irq/manage.c` `request_threaded_irq()` `irq_thread_fn()` threaded irq

13.7.3 “ + softirq”

```

IRQ
gic_handle_irq
-> generic_handle_domain_irq
-> handle_fasteoi_irq / handle_percpu_devid_irq

irq_exit_rcu
-> invoke_softirq
-> __do_softirq
-> softirq action

```

13.8 中断亲和性

“软中断”`softirq` 中断亲和性

1. 中断亲和性 CPU
2. 中断亲和性 CPU
3. 中断亲和性 CPU

中断亲和性 `affinity` `softirq` 中断亲和性

13.9 中断亲和性

ARM64/GICv3 中断亲和性 “IRQ 中断”`timer` 中断亲和性
watchdog 中断

13.9.1 中断

1. 中断亲和性
2. `ksoftirqd` `timer` `RCU` 中断亲和性
3. `/proc/interrupts` 中断亲和性
4. CPU 中断 trace 中断亲和性 `irq-off` 中断
5. watchdog 中断亲和性 `latency/warn` 中断

13.9.2 中断

“中断亲和性”`local_irq_disable()` 中断亲和性

1. 中断亲和性 `IRQ` 中断亲和性
2. 中断亲和性 CPU 中断亲和性
3. `timer tick` `IRQ` `IPI` 中断亲和性

中断亲和性 `IRQ-off latency` 中断

13.9.3 中断

1. `spin_lock_irqsave()` / `raw_spin_lock_irqsave()` 中断亲和性
2. 中断亲和性
3. `hardirq` 中断亲和性
4. 中断亲和性 `threaded irq` / `workqueue` / `softirq` 中断亲和性 `hardirq` 中断亲和性 `irq-off` 中断
5. `pseudo-NMI` `IPI` `stop machine` `CPU hotplug` 中断亲和性

13.9.4 中断亲和性 `irqsoff tracer`

中断亲和性 `/proc/interrupts` 中断亲和性 `ftrace` 中断亲和性 `irqsoff tracer`

中断亲和性 `Documentation/trace/ftrace.rst` 中断亲和性 `irqsoff` 中断亲和性 “中断亲和性”

中断亲和性

```

echo 0 > /sys/kernel/tracing/options/function-trace
echo irqsoff > /sys/kernel/tracing/current_tracer
echo 0 > /sys/kernel/tracing/tracing_max_latency
echo 1 > /sys/kernel/tracing/tracing_on

# 断点

echo 0 > /sys/kernel/tracing/tracing_on
cat /sys/kernel/tracing/trace

```

断点 `function_graph` 断点 `IRQ-off`

13.9.5

断点

1. `raw_spin_lock_irqsave()` / `spin_lock_irqsave()` 断点
2. `gic_handle_irq()` 断点 `primary handler`
3. GIC/ITS 断点
4. `hardirq/top-half` 断点

13.9.6

断点

1. `irq-off` 断点
2. `threaded irqsoftirqtasklet` `workqueue`
3. “断点”断点
4. 断点
5. `raw_spin_lock_irqsave()` `local_bh_disable()`

13.9.7

1. `Documentation/trace/ftrace.rst irqsoff` `tracer` 断点
2. `kernel/trace/trace_irqsoff.c` `IRQ-off latency` 断点
3. `kernel/irq/manage.c` `IRQ` 断点
4. `kernel/irq/chip.c` `flow handler` 断点

13.9.8 QEMU “断点”

断点 “断点”

断点 4 vCPU 断点 CPU `irq-off` 断点

```

qemu-system-aarch64 \
-M virt,gic-version=3 \
-cpu cortex-a57 \
-smp 4 \

```

```
guest [irqsoff] tracer
```

□ □ □ □ □ □ □ □ □

- 111

```
□□□□□ shell□
```

tracing

□□□□□□

- “”`local_irq_disable()` busy loop
`irqsoff` tracer

13.10 中断处理 loop 和 pending

中断“loop”和 pending 是中断处理中的两个重要概念。

1. handler 中断处理函数
2. 中断/事件 pending 标志
3. edge 敏感 flow handler 中断处理函数
4. 硬中断 hardirq 和 loop 中断“loop”

13.10.1 中断 loop

中断“loop”是指中断处理函数在返回后，再次被调用的现象。

1. **flow handler** 中断敏感 `handle_edge_irq()` 标志 `IRQS_PENDING` 标志 `do ... while` 标志
2. **resend / repeated pending** 标志 `handle_fasteoi_irq()` 标志 `IRQS_PENDING` 标志 `check_irq_resend()`
3. 中断敏感 标志 `asserted` CPU 标志“中断”

13.10.2 中断 GICv3 中断处理

中断 GICv3 标志 `handle_fasteoi_irq()` 标志 `handle_edge_irq()` 标志 `do/while` 标志

1. 中断 ISR 中断处理函数
2. level 标志 `active/asserted`
3. affinity 标志 `resend` 标志 `pending`
4. threaded irq / primary handler 标志 `IRQ core` 标志

13.10.3 中断标志

1. `/proc/interrupts` 标志 `IRQ` 标志
2. 标志 `CPU` 标志 `hardirq` 标志
3. 标志 `irq X: nobody cared`
4. 标志 `spurious` 标志
5. 标志 `CPU` 标志 `handler` 标志

13.10.4 中断

中断标志 `tracepoint`

```
cat /proc/interrupts
cat /proc/irq/<irq>/spurious
cat /sys/kernel/tracing/trace_pipe
```

中断 `tracepoint`

□ □ □ □ □ □ □ □

- ### 13.10.5

□ □ □ □ □ □ □ □

- “CPU □□□”□□□ IRQ core □□□□□□□□□□

13.10.6 练习

- ### 13.10.7 〇〇〇〇〇〇

□ □ □ □ □ □ □ □ □ □ □ □

1. 指定 trigger type
2. 指定 handler
3. 指定 IRQ core
4. 指定 handler 的 IRQ 号
5. 指定 handler 的 name

```
handler
```

13.10.8 GICv3

1. GIC 优先级 drop / deactivate 操作
2. 中断屏蔽 level 操作
3. 中断“GIC 操作”操作

1. GIC 寄存器
2. 寄存器地址
3. handler 函数

13.10.9 □□□□

- ### 13.10.10 QEMU “loop / loop”

“IRQ softirq”

guest

55 / 59

```
echo 1 > /sys/kernel/tracing/events/irq/softirq_exit/enable
cat /sys/kernel/tracing/trace_pipe
```

guest I/O NET_RX

1. IRQ
2. IRQ raise softirq
3. CPU
4. /proc/interrupts /proc/softirqs

“loop”

1. level
2. handler hardirq trace

13.11

“”

1. irq-off
- 2.

“IRQ CPU”

- 1.
2. trigger type
3. handler

13.12

13.12.1 “”

1. arch/arm64/kernel/entry.S vectors kernel_ventry entry_handler ellh_64_irq_handler
2. arch/arm64/kernel/entry-common.c ell_interrupt()
3. drivers/irqchip/irq-gic-v3.c gic_handle_irq()
4. kernel/irq/irqdesc.c generic_handle_domain_irq() / generic_handle_irq_desc()
5. kernel/irq/chip.c handle_fasteoi_irq() flow handler

“CPU ISR”

13.12.2 “flow handler” handler

概要

1. `kernel/irq/chip.c handle_fasteoi_irq()`
2. `kernel/irq/manage.c handle_irq_event()` `handle_irq_event_percpu()`
3. `kernel/irq/manage.c __handle_irq_event_percpu()`
4. `request_irq()` / `request_threaded_irq()` から ISR

“IRQ core” “IRQ core”

13.12.3 “”

概要

1. `kernel/irq/handle.c irq_exit_rcu()`
2. `kernel/softirq.c invoke_softirq()`
3. `kernel/softirq.c __do_softirq()`
4. `kernel/softirq.c` softirq action timer tasklet

“hardirq” “softirq”

13.12.4 “ / / spurious / resend”

概要

1. `drivers/irqchip/irq-gic-v3.c gic_set_affinity()`
2. `drivers/irqchip/irq-gic-v3-its.c its_set_affinity()`
3. `kernel/irq/chip.c handle_edge_irq()` `handle_fasteoi_irq()`
4. `kernel/irq/resend.c check_irq_resend()`
5. `kernel/irq/spurious.c note_interrupt()`

“flow handler”

13.12.5

概要

- 1.
- 2.
- 3.

GICv3 “”

14

1 ISR

```

el1h_64_irq_handler
-> el1_interrupt
-> handle_arch_irq
-> gic_handle_irq
-> generic_handle_domain_irq
-> handle_irq_desc
-> handle_fasteoi_irq / handle_percpu_devid_irq
-> [] handler

```

2 GICv3 の初期化

```

DT/ACPI
-> gic_of_init / gic_acpi_init
-> gic_init_bases
-> domain []
-> GICD []
-> CPU GICR []
-> ICC sysreg []
-> SMP/IPI []
-> ITS []

```

3 ITS の初期化

```

MSI write
-> ITS_TRANSLATER
-> DeviceID/EventID
-> ITT
-> LPI
-> Collection
-> Redistributor
-> CPU

```

このガイドは、GICv3 の初期化と IRQ core の構成について説明します。

1 GICv3 の初期化

1 GICv3 の初期化は、以下の手順で行われます。

まず GICD、GICR、ICC sysregs、ITS の初期化は Linux の `irq_domain` と `irq_chip` を利用して行われます。

2 次に GICv3 の IRQ core の構成を行います。

Linux の `ICC_IAR1_EL1` と `ICC_EOIR1_EL1` は、

1. `irq_domain` の初期化
2. `irq_desc` の flow handler の設定

3. `irq_chip` 関数 mask/unmask/eoi/deactivate

3. GICv3 対応デバイス SPI/PPI ITS/LPI/GICv4

対応デバイス

- 1. ITS デバイス
- 2. LPI デバイス
- 3. vLPI/vPE デバイス
- 4. direct LPI doorbell VMOPV デバイス

4. 対応デバイス

- 1. INTID デバイス
- 2. SGI cluster デバイス
- 3. LPI first-fit デバイス
- 4. ITS ring buffer デバイス

5. 対応デバイス “デバイス” “デバイス”

デバイス

- 1. デバイス
- 2. デバイス
- 3. デバイス
- 4. trace
- 5. デバイス

デバイス

デバイス

デバイス

- 1. デバイス `irq-gic-v3.c` デバイス
- 2. `irq_domain` `handle_fasteoi_irq` デバイス
- 3. デバイス `irq-gic-v3-its.c` デバイス **5000** デバイス
- 4. デバイス **GICv4** デバイス **host IRQ API** “” **hypervisor** デバイス

デバイス Linux ARM64 GICv3 デバイス